Knowledge Distillation and Deployment of a Lightweight Command-Parsing LM on a Local Cloud

CS3081 Cloud Computing mini project report

Vikas Pandey | Roll no. **230101004** | Section A | B.Tech CSE, Indian Institute of Information Technology

Repository: https://github.com/vikas1311code/cloud-kd-command-parser (final commit tagged **final**). Evidence files are in **results/**. Build, deploy, test and shutdown commands are in README.md.

How to read this report. Every number marked **Measured** comes from a run on my own laptop and the file that holds it is named next to it. Statements marked **Assumption** or **Interpretation** were not measured. No result was copied from anywhere else. The machine is a Windows laptop running Docker Desktop and k3d, so all latency and cost figures describe that setup only.

Task 1. Storage and parallel labelling

Architecture. One k3d cluster (*cloudlab*, cluster/k3d.yaml). SeaweedFS runs in namespace *data* (k8s/seaweedfs.yaml, S3 API on port 8333, 5 Gi volume) with buckets *datasets* and *models*. Labelling workers run as Kubernetes jobs in namespace *pipeline* (k8s/worker-jobs.yaml). Each worker reads commands_pool.jsonl from the bucket *datasets* and keeps the lines whose index modulo SHARD_COUNT equals its SHARD_INDEX (round-robin sharding). It sends each command to the teacher (phi3.5 on Ollama, temperature 0, seed 42, num_predict 120) and writes the raw replies to datasets/teacher/shard-<k>.jsonl; commands already written are skipped, so a restarted worker resumes. Two workers ran (shard-0 and shard-1). The prompt is prompts/teacher_prompt.txt.

Implementation. worker/clean.py reads the raw replies and keeps a reply only if it is valid JSON with exactly the four keys intent, date, time, place, an intent from the allowed list in data/schema.json, and slot values that really occur in the command. It also removes duplicates, writes datasets/v1/train.jsonl and its SHA-256, and reports counts. Unit tests: tests/test_clean.py (6 tests).

Measured: results/clean_counts.json and clean_run.log.

Input records	Rejected (a) invalid JSON	(b) wrong keys	(c) bad intent	(d) bad slot	Duplicates	Kept	Missing / repeated ids
3,000	303	1	2	81	42	2,571	0 / 0

train.jsonl SHA-256: 110a92f3c318f4ecb0f283ee196ddd30b2c79fa342e6b74a8100dba524128612. The check 3,000 = 303 + 1 + 2 + 81 + 42 + 2,571 holds. **Limitations.** The labelling time was not recorded in results/, so I make no claim about speed-up from two workers. Cleaning checks format and copying only; it cannot detect a reply that is valid but wrong (see Task 2). **Improvements.** Record per-shard timings, add a semantic check against a small rule set, and use more than two workers if the teacher host has spare cores.

Task 2. Student training (sequence-level distillation)

Architecture and implementation. The student is EleutherAI/pythia-70m. For each command the input text is "Command: <text>\nJSON:" and the target is the teacher's JSON, trained with ordinary next-token cross-entropy on the **first 1,500 cleaned records** (training log: first id C0001, last id C1720; ids up to C1720 because cleaning removed some of the first replies). Training runs on CPU inside the container built from training/Dockerfile via *make train* (torch 2.5.1+cpu, transformers 4.46.3, numpy 1.26.4). Settings (training/config.yaml): learning rate 1e-4, batch size 8, weight decay 0.01, warm-up 10 steps, max sequence length 128, seed 42, 3 epochs for v1. v2 (training/config_v2.yaml) changes the seed to 43 and the epochs to 4.

Measured: results/loss.csv, loss_v2.csv, train_subset50.txt.

Run	Epoch 1	Epoch 2	Epoch 3	Epoch 4
v1 (seed 42)	0.3236	0.0976	0.0440	-
v2 (seed 43)	0.3106	0.1170	0.0575	0.0235
make train SUBSET=50 (50 records)	1.7349	0.3912	0.1590	-

The loss falls in every epoch, which shows the student fits the teacher's text. Loss on the training set alone does not show generalisation; that is what the validation score below is for.

Measured: model and release checks (results/model_v1.sha256, model_v2.sha256, release_checksum_check.txt, version_info_v*.json). model.safetensors is 281,715,176 bytes for both versions. SHA-256 v1 e808f7d9...30bc1, v2 70cf279e...92e6f6 (different, so two real models). The files downloaded from the GitHub Releases model-v1 and model-v2 have the same size and the same hash as the local files (result: MATCH for both). Model files are not in Git; they are Release assets.

Measured: validation on the 100 labelled commands with tools/score.py (results/score_*.json, task2_scores.md). All values in percent.

Model	json_validity	intent	date	time	place	full_frame
Teacher (phi3.5)	84.0	76.0	69.0	77.0	77.0	50.0
Untrained Pythia-70M	0.0	0.0	0.0	0.0	0.0	0.0
Student v1	100.0	87.0	81.0	90.0	85.0	52.0
Student v2	100.0	86.0	78.0	91.0	85.0	53.0

Interpretation (not measured). The student produces valid JSON on all 100 examples because it was trained only on cleaned, valid replies, while the teacher is valid on 84. The student's full-frame score (52 or 53) is close to the teacher's 50. With only 100

validation examples a 2 to 3 point gap is about 2 to 3 examples, so I do not claim that the student is better than the teacher, or that v2 is better than v1. **Why sequence-level and not logit-based:** only the teacher's text is stored, not its token probabilities, and phi3.5 and Pythia use different tokenizers, so token positions cannot be matched (full text in report/task2_explanation.md). **Limitation found:** the student copies the teacher's mistakes. For V003 ("give me the time at washington right now") the teacher put "washington" in the time slot instead of place, and student v1 repeated it. **Improvements.** Correct or filter the teacher labels (for example with a second check or small hand-labelled set), and train on all 2,571 cleaned replies once the 1,500 limit no longer applies.

Task 3. Inference service

Architecture and implementation. service/app.py is a FastAPI app. On start a background thread downloads the model prefix given by MODEL_URI (for example s3://models/v1) from SeaweedFS with credentials from the environment, loads the tokenizer and model, and only then sets ready (so /readyz is not ready while loading). Endpoints: **/healthz** (alive), **/readyz** (ready after the model is loaded), **/parse** (POST {"text": ...}, returns the four keys). Every /parse response carries **X-Model-Version** and **X-Served-By** (pod name). If the model output is not valid JSON the service returns 500 with {"error":"invalid_model_output"}. Setting FAULT_INJECT=1 forces that failure. The image (service/Dockerfile) uses CPU-only torch.

Measured: results/task3_*.txt.

Check	Result
/healthz, /readyz, /parse	all HTTP 200; /parse returned {"intent":"alarm_set","date":"tomorrow","time":"seven thirty am","place":null} with x-model-version: v1
FAULT_INJECT	HTTP 500, body {"error":"invalid_model_output"}
Image size	kd-service:v1: 1.6 GB on disk (341 MB content size), under the 2 GB limit
50 requests at 1 CPU / 1 GiB	50 / 50 returned 200; median 0.47 s, p95 0.55 s, max 1.14 s; ready after 18 s
Memory	peak 391.9 MiB seen in 22 docker stats samples (limit 1,024 MiB); OOMKilled=false, restarts 0

Limitations. The peak memory comes from sampling with docker stats; cgroup memory.peak was not available, so a short spike between samples could be missed. The 0.47 s median is for a single client; under 4 concurrent users the median was 1.4 s (Task 7). **Improvements.** Stop generation at the closing brace instead of always allowing 64 new tokens, validate the output against schema.json before returning, and export the model to a faster runtime.

Task 4. Kubernetes

Architecture. Namespace *app*. **Deployment** kd-parser (k8s/deployment.yaml): 2 replicas, RollingUpdate with maxUnavailable 0 and maxSurge 1, requests 500m CPU / 512 Mi, limits 1 CPU / 1 Gi, liveness probe /healthz (initial delay 10 s, period 10 s), readiness probe /readyz (period 3 s), preStop sleep 5 s. S3 credentials come from the secret s3-credentials. **Service** kd-parser maps port 80 to container port 8000. **Ingress** routes path /parse to the Service (k3d maps host port 8081 to the cluster load balancer). **HPA** (k8s/hpa.yaml): 1 to 2 replicas, target 70 % CPU of the request, scale-down stabilisation 60 s. Scripts: scripts/task4_*.py.

Measured: results/task4_*.txt and loadtest_*.csv.

Test	Result
Cold start (delete all pods, time to Ready, 3 runs)	14 s, 17 s, 17 s; median 17 s (task4_cold_start.txt)
Not Ready while the model loads	endpoints list stayed empty until 13 s after start, then both pod IPs appeared (task4_endpoints_while_loading.txt)
Load balancing through the Ingress	276 successful requests: 137 (49.6 %) and 139 (50.4 %) on the two replicas, all model v1 (task4_load_balancing_shares.txt)
Pod delete under load (4 users, 2 min)	replica count restored after 17 s; 262 requests, 0 failures (task4_pod_delete_log.txt)
Rolling update v1 to v2, then rollback (4 users, 4 min)	update finished in 39 s, probes then returned v2; rollback finished in 35 s, probes returned v1; 480 requests, **2 failures (0.417 %)**; responses: 352 from v1, 128 from v2 (task4_rolling_update_log.txt)
HPA scale-out and scale-in	HPA events show size 1 to 2 ("cpu above target") and later 2 to 1 ("all metrics below target") (task4_hpa_describe.txt, task4_hpa_watch.txt)
Load test with HPA (standard locust command)	467 requests, 0 failures, median 1,500 ms, p95 2,300 ms, 2.61 req/s (loadtest_scale_stats.csv)

The update to v2 was made by changing MODEL_VERSION and MODEL_URI on the Deployment, so a new Pod downloads the other model version from the bucket; the image was not rebuilt. **Limitations.** (1) The rolling update was not fully error-free: 2 of 480 requests failed. I did not diagnose the cause, so I cannot say whether it happened during the update, the rollback, or at the start; zero downtime is therefore **not** claimed. (2) The HPA log also shows early warnings "did not receive metrics for targeted pods" while the pods were starting. (3) Several early scale-test attempts did not finish and are kept as results/attempt1 to attempt4 files (aborted run, all-503 responses, a stuck Pod, Locust not started) rather than hidden. (4) 17 s cold start is long because every Pod downloads the 281 MB model. **Improvements.** Add a startup probe, cache the model on a shared volume, keep minReplicas at 2 for availability, and add a retry on the client side during rollouts.

Task 5. Subnetting and network ACL

Subnets for 10.20.0.0/16 (network/subnets.md). For each requirement I used the smallest prefix whose usable hosts (2^(32-prefix) - 2) are enough, allocated largest first, and cross-checked with Python's ipaddress module.

Subnet	Hosts needed	Prefix	Network	Usable range	Broadcast	Usable hosts
app	1,000	/22	10.20.0.0	10.20.0.1 - 10.20.3.254	10.20.3.255	1,022

Subnet	Hosts needed	Prefix	Network	Usable range	Broadcast	Usable hosts
public	250	/24	10.20.4.0	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data	100	/25	10.20.5.0	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management	20	/27	10.20.5.128	10.20.5.129 - 10.20.5.158	10.20.5.159	30

NACL for the public subnet is in network/nacl.json (network ACLs are stateless, so replies need their own rules). It is checked with tools/nacl_sim.py. **Measured** (results/nacl_check.txt): the 6 given sample packets were decided correctly (6 / 6 = 100 %), and my own 20 packets (network/packets_extra.jsonl) were also decided as I expected (20 / 20 = 100 %), including blocked-range, SSH-only-from-management, port-boundary (1023 / 1024), UDP and default-deny cases. Tests: tests/test_nacl.py (6 tests). **Limitation.** The simulator checks my rules against my own expectations; it is not a real cloud firewall, and nothing here was applied to a real VPC. **Improvement.** Add security-group style stateful rules for comparison and test more boundary packets automatically.

Task 6. Reproducibility, testing and CI/CD

make up / make down. From a fresh clone: *pip install -r requirements-dev.txt* then *make up* (two commands; the limit is three). scripts/make_up.py creates the cluster, namespaces and a random S3 secret (or reuses an existing one, so no password is stored in Git), applies SeaweedFS, creates the buckets, downloads the model assets from the GitHub Release with resume and retry, verifies every file against model.sha256 (and re-downloads a file whose hash differs) before uploading to the bucket, builds and imports the image, deploys, and checks /readyz and /parse. *make down* deletes the cluster.

Measured: results/make_up_*.txt. Clean test: *make down*, new clone in an empty folder, then the two commands. It finished in **456 s**, both kd-parser Pods were 1/1 Running, /readyz returned 200, and a request through the Ingress (http://localhost:8081/parse) returned 200 with X-Model-Version v1 (make_up_fresh_clone.txt, _pods.txt, _ingress.txt). **Earlier attempts failed and are kept:** attempt 1 stopped after 175 s (HTTP 504 from GitHub, then a crash on a missing file); attempt 2 stopped after 23 s with a checksum mismatch on one file (the checksum guard blocked a bad file from reaching the bucket); attempt 3 succeeded on a warm cluster in 554 s. Each failure led to a fix in the download code (curl with -f and resume, restart of small files, re-download on mismatch).

CI (.github/workflows/ci.yml): jobs *lint* (ruff), *test* (18 unit tests), *evaluate* (downloads model v1 from the Release, checks the checksum, runs eval/gen_student.py and tools/score.py on the validation set), *gate* (tools/score.py --min-validity 95), and a separate *secrets* job (gitleaks over the full history). A manual run can choose v1, v2 or the untrained model.

Measured: results/ci_runs.txt. Run #1 passed all jobs (https://github.com/vikas1311code/cloud-kd-command-parser/actions/runs/42462903229). Run #2 used the untrained Pythia-70M on purpose: lint, test, evaluate and secrets passed and **gate failed** (https://github.com/vikas1311code/cloud-kd-command-parser/actions/runs/42463158533). Runs #3 to #10 (later commits) all succeeded when listed. Gitleaks reported nothing. ruff and the 18 unit tests also pass locally. AI use is described in AI_USE.md.

Limitations. The evaluation job re-scores only the student on the validation set; it does not redeploy the cluster, so Kubernetes behaviour is tested only by my local scripts. make up needs about 7 to 9 minutes and a working internet connection to GitHub. **Improvements.** Add a smoke test with k3d to CI, cache the pip and model downloads, and pin the GitHub Action versions by commit.

Task 7. Performance, cost and review

Setup. Standard Locust command (4 users, spawn rate 4, 2 minutes, tools/locustfile.py) against the Ingress at http://localhost:8081 (the standard command names port 8080; I used the Ingress so both replicas share the load). 2 replicas, model v1. CPU and memory from *kubectl top pods* sampled every 25 s.

Measured: results/task7_loadtest_*.csv, task7_loadtest_console.txt, task7_top_pods.txt, task7_analysis.md.

Measurement	Value
Requests / failures / error rate	315 / 0 / 0.00 % (limit 1 %)
Latency p50 / p95 / p99 / max	1,400 ms / 2,300 ms / 2,400 ms / 3,320 ms (limit for p95: 5,000 ms); average 1,508 ms
Throughput	2.64 requests per second
CPU per Pod (4 samples while loaded)	777m to 999m of a 1,000m limit (both Pods at about 999m in the last 3 samples)
Memory per Pod	358 Mi to 520 Mi of a 1 GiB limit
Cold start (Task 4)	median 17 s (14, 17, 17 s)
Requests per replica	195 and 120 (61.9 % and 38.1 %)

Cost per 1,000 requests (data/rate_card.json: 3.50 INR per vCPU-hour, 0.45 INR per GiB-hour; a replica is billed for its limits, 1 CPU and 1 GiB, for every hour it runs). (1) One replica per hour = 1 x 3.50 + 1 x 0.45 = 3.95 INR. (2) Two replicas = 2 x 3.95 = 7.90 INR per hour. (3) Requests per hour = 2.6408 x 3,600 = 9,506.9. (4) Cost per 1,000 requests = 7.90 / 9,506.9 x 1,000 = **0.831 INR**. **Assumption:** the measured 2.64 req/s would hold for a full hour; no real cloud charge is made.

SLO. Target: 99 % of /parse requests return success. Window: one load-test run (315 requests). Error budget = 1 % x 315 = 3.15 requests (at most 3 failures). Used: 0 failures, so 0 of 3.15 of the budget (0 %). (The rolling-update run in Task 4 had 2 failures in 480 requests, 0.417 %, which would use about 2 of its 4.8 allowed failures, but that run is a different window.)

Review table (every Evidence cell holds a measured number).

Concern	Limitation found	Improvement proposed	Evidence (measured)
Data and storage	One SeaweedFS replica; deleting the cluster deletes the bucket	Keep models in Releases (done), re-seed on make up; replicated storage in a real cloud	model.safetensors 281,715,176 bytes; full rebuild by make up 456 s
Model quality	Student copies teacher mistakes (V003)	Filter or correct labels, add data	student v1 full_frame 52.0 vs teacher 50.0; date accuracy 81.0
Latency and throughput	CPU-only generation of up to 64 tokens	Stop at closing brace, faster runtime, batching	p50 1,400 ms, p95 2,300 ms, 2.64 req/s
Resource utilisation	CPU at the limit under load, memory about half used	More CPU or replicas, lower memory request	CPU 999m of 1,000m; memory 520 Mi of 1,024 Mi
Reliability and recovery	Slow cold start; 2 failures during update and rollback	Startup probe, model cache, minReplicas 2	cold start median 17 s; 2 of 480 requests failed
Security and cost	Billed for limits even when idle; secrets must stay out of Git	Scale down when idle; random secret per cluster and gitleaks (done)	0.831 INR per 1,000 requests; gitleaks job reported nothing

Summary of what is measured and what is assumed

Measured: all counts, losses, hashes, scores, latencies, memory and CPU values, cold-start times, load shares, CI results and make up timings above, each from a file in results/. **Assumed or interpreted:** the one-hour extrapolation in the cost estimate, the rate card prices (hypothetical), the reading of small score differences, and the explanation of why the student copies teacher errors. **Not done or not diagnosed:** labelling speed-up, the cause of the 2 failed requests in the rolling update, and any run on a real cloud.